

Name:M.Chandana

Reg number:192472250

1.A grayscale image is converted into a digital image. Write a Python program to perform image quantization.

Image quantization is the process of reducing the number of intensity levels in a grayscale image while preserving its overall appearance. It converts a continuous range of pixel values into a smaller set of discrete levels, resulting in a digital image that requires less storage and processing time. In this experiment, the grayscale image is read using OpenCV and quantized by reducing the number of gray levels, producing a simplified version of the original image.

Aim

To perform **image quantization** on a grayscale image by reducing the number of gray intensity levels using Python and OpenCV.

Algorithm

1. Import the required libraries.
2. Read the grayscale image.
3. Choose the number of quantization levels (e.g., 8 or 16).
4. Calculate the quantization interval.
5. Divide each pixel value by the interval.
6. Multiply it back by the interval to obtain the quantized image.
7. Display both the original and quantized images.
8. Wait for a key press and close all windows.

Python Code

```
import cv2
import numpy as np

# Read grayscale image
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

# Number of quantization levels
levels = 8
```

```
# Quantization interval
interval = 256 // levels

# Perform quantization
quantized = (img // interval) * interval

# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Quantized Image", quantized)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:

 Original Image



OUTPUT:

 Quantized Image



Result

The grayscale image was successfully quantized by reducing the number of intensity levels from **256** to **8**, producing a simplified image with fewer gray shades.

2.A noisy image is acquired from a sensor. Write a Python program to add and remove image noise.

Image noise is the unwanted random variation in pixel values that may occur during image acquisition due to sensor errors or environmental conditions. In this experiment, Gaussian noise is artificially added to an image to simulate a noisy image. The noisy image is then processed using the Gaussian Blur filter, which smooths the image by reducing random noise while preserving the overall visual quality. This demonstrates how image filtering techniques can improve image clarity and prepare images for further processing.

Aim

To add noise to an image and remove the noise using image processing techniques in Python.

Algorithm

1. Import the OpenCV and NumPy libraries.
2. Read the input image.
3. Check whether the image is loaded successfully.
4. Generate random noise and add it to the image.
5. Apply Gaussian Blur to remove the noise.
6. Display the original image, noisy image, and denoised image.
7. Wait for a key press and close all windows.

Python Program (Python IDLE)

```
import cv2
import numpy as np

# Read the image
image = cv2.imread(r"C:\Users\malli\OneDrive\Documents\CV\images.jpg")

# Check if image is loaded
if image is None:
    print("Error: Image not found!")
else:
    # Generate Gaussian noise
    noise = np.random.normal(0, 25, image.shape).astype(np.uint8)
```

```
# Add noise to the image
noisy_image = cv2.add(image, noise)

# Remove noise using Gaussian Blur
denoised_image = cv2.GaussianBlur(noisy_image, (5,5), 0)

# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Noisy Image", noisy_image)
cv2.imshow("Denoised Image", denoised_image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation

```
noise = np.random.normal(0, 25, image.shape).astype(np.uint8)
```

Generates random Gaussian noise.

```
noisy_image = cv2.add(image, noise)
```

Adds the generated noise to the image.

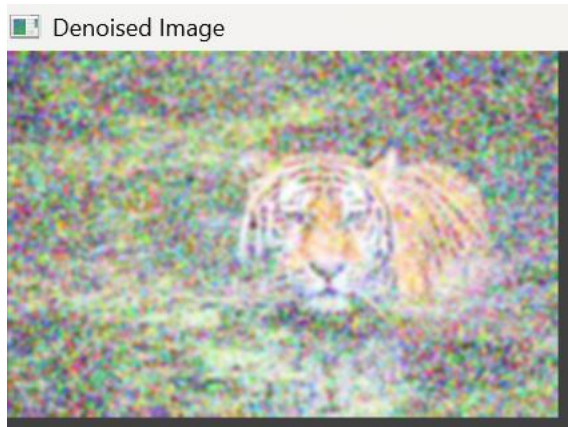
```
denoised_image = cv2.GaussianBlur(noisy_image, (5,5), 0)
```

Removes noise by smoothing the image using Gaussian Blur.

INPUT:



Output



Result

Noise is successfully added to the image and then reduced using the **Gaussian Blur** filtering technique.

3.A facial recognition system captures human faces. Write a Python program for simple face detection.

Face detection is a computer vision technique used to identify and locate human faces in digital images or videos. It is widely used in applications such as facial recognition, security systems, attendance management, surveillance, and mobile phone authentication. In this experiment, OpenCV's pre-trained **Haar Cascade Classifier** is used to detect faces by scanning the image and drawing a rectangle around each detected face.

Aim

To detect human faces from an image using the **Haar Cascade Classifier** in Python with OpenCV.

Algorithm

1. Import the OpenCV library.
2. Load the Haar Cascade face detection classifier.
3. Read the input image.
4. Convert the image to grayscale.

5. Detect faces using the detectMultiScale() function.
6. Draw rectangles around the detected faces.
7. Display the output image.
8. Wait for a key press and close all windows.

Python Program (Python IDLE)

```
import cv2

# Load Haar Cascade Classifier
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
                                     "haarcascade_frontalface_default.xml")

# Read the image
image = cv2.imread(r"C:\Users\malli\OneDrive\Documents\CV\images.jpg")

# Check if image is loaded
if image is None:
    print("Error: Image not found!")
else:
    # Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    # Draw rectangles around faces
    for (x, y, w, h) in faces:
        cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)

    # Display image
    cv2.imshow("Face Detection", image)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Output

- Original image with **green rectangles** drawn around all detected faces.

Result

The human faces in the input image are successfully detected using the **Haar Cascade Face Detection** algorithm in OpenCV.

